Algorithm Design Paradigms

Handout on Wrong Pseudocodes

13 February 2026

§1 Basic Stringology

? Problem

We will call a string consisting of characters 0, 2, 5, and/or 6 a *New Year* string if at **least** one of the two conditions is met:

- it contains the string 2026 as a continuous substring;
- it does not contain the string 2025 as a continuous substring.

For example, the strings 20252026, 21026, 20262026, 000 are New Year strings. The strings 2025, 20256, 20252025, 000202500020226 are not New Year strings.

Write code that given a string determines if it is *New Year* in $O(n)$ time and $O(1)$ space.

Wrong Pseudocode

```
function isNewYear(s):
    found2026 ← false
    found2025 ← false
    state26 ← 0
    state25 ← 0
    for each character c in s:
        if state26 == 0 and c == '2':
            state26 ← 1
        else if state26 == 1 and c == '0':
            state26 ← 2
        else if state26 == 2 and c == '2':
            state26 ← 3
        else if state26 == 3 and c == '6':
            found2026 ← true
            state26 ← 0
        else:
            state26 ← 0
        if state25 == 0 and c == '2':
            state25 ← 1
        else if state25 == 1 and c == '0':
```

Wrong Pseudocode

```
                    state25 ← 2
                else if state25 == 2 and c == '2':
                    state25 ← 3
                else if state25 == 3 and c == '5':
                    found2025 ← true
                    state25 ← 0
                else:
                    state25 ← 0
        if found2026 or not found2025:
            return true
        else:
            return false
```

(a) The algorithm described is wrong and fails for some test case. Can you find one?

(b) Can you find where did it go wrong?

(c) Write the correct $O(n)$ time and $O(1)$ space algorithm and prove it's correctness and complexity.

(d) Modify the code to modify the string with minimal number of changes (turn one character to another another from $0, 2, 5, 6$) to make the string *New Year* in $O(n)$ time and $O(1)$ space.

? Problem

We will call a string consisting of characters 's' and 'u', *SUS* if **both** the following conditions are met:

- The letter 's' appears at least twice
- For every occurrence of the letter 'u', the two nearest occurrences of 's' are the same number of characters away from the 'u'

For example: "sus", "sssss", "ssssss" are *SUS* but "uuuu", "susu" "uusuuu", "susuusuuus" are not.

Write code that given a string determines if it is *SUS* in $O(n)$ time and $O(1)$ space.

Wrong Pseudocode

```
def sus(input):
    countS ← 0
    for i in input:
        if i == 's': countS += 1
    if countS < 2:
        return False
    i = 0
    j = 0
    k = 0
    dist = −1
```

Wrong Pseudocode

```
n = length(input)
while j < n:
    if input[k] == 'u' || j >= k
        k += 1
    if input[j] == 'u'
        if j - i != k - j
            return False
        if dist = −1:
            dist = j - i
        else:
            if dist != j - i:
                return False
    else:
        i = j
        j += 1
```

(a) The algorithm described is wrong and fails for some test case. Can you find one?

(b) Can you find where did it go wrong?

(c) Write the correct $O(n)$ time and $O(1)$ space algorithm and prove it's correctness and complexity.

(d) Could you notice something about *SUS* strings to make the code cleaner?

? Problem

We call a string s consisting of characters 'a', 'b' and 'c' *fancy* if one can't find i, j, k such that $s[i, j] = s[j, k]$ and $j - i = 1$ or 2.

For example: "abc", "abcba", "abcbac", "abcabc" are *fancy* while "aab", "ababc", "abccba" are not.

(a) Write the correct $O(n)$ time and $O(1)$ space algorithm and prove it's correctness and complexity.

(b) If you look at the solutions to the last 3 problems, you might see some similarity. Could you describe it?

§2 Triangles

? Problem

A finite, at least three-element list A of rational-length segments is given. We want to examine whether a triangle may be created from every three segments in A. (that is $\forall x, y, z \in A; \exists$ a triangle with side x, y, z).

For example: $\left[\frac{13}{10}, \frac{1}{2}, \frac{6}{5}, \frac{11}{6}, \frac{9}{7}, \frac{3}{5}, \frac{9}{7}, \frac{13}{10}, \frac{9}{5}, \frac{8}{5}\right]$ would return **false** as $\frac{3}{5}, \frac{6}{5}, \frac{9}{5}$ don't form a triangle.

On the other hand, $\left[\frac{1}{2}, \frac{3}{5}, \frac{2}{3}, \frac{4}{7}, \frac{1}{1}, \frac{4}{6}\right]$ would return **true**.

Wrong Pseudocode

```
1  fn Trig(A : List Rational) {
2      flag = true;
3      for a in A{
4          for b in A{
5              for c in A{
6                  if (a + b > c) && (a + c > b) && (b + c > a){
7                      continue
8                  } else {flag = not flag}
9              }
10         }
11     }
12     return flag
13 }
```

(a) The algorithm described is wrong and fails for some test case. Can you find one?

(b) Can you find where did it go wrong?

(c) Modify the algorithm and analyze it's time complexity.

(d) Can you find an algorithm which solves this in $O(n)$ time?

(e) ♠ This problem appeared on a programming contest and later in Unity interviews, and remains mostly unsolved (Unity being one of the companies is a hint). Could you guess why? Here are the original constraints:

Input: The rationals will have numerators and denominators $< 10^4$

Computer: The computer was 32-bit linux

(f) ♠ Can you figure out a way to solve it under original constraints?

§3 Genome

? Problem

You are given two strings, `Ideal` and `Patient`, representing genetic codes.

`Patient` is obtained from `Ideal` by deleting exactly k characters.

Determine the indices (with respect to `Ideal`) and values of the deleted characters.

Wrong Pseudocode

```
class Deletion:
    def init(self, index, value):
        self.index = index
        self.value = value
def find_deletions(ideal, patient, k):
    results = []
    p1 = 0
    p2 = 0
    while p1 < len(ideal):
        if p2 < len(patient) and ideal[p1] == patient[p2]:
            p1 += 1
            p2 += 1
        else:
            results.append(Deletion(p1, ideal[p1]))
            p1 += 1
            p2 += 1
    return results
```

(a) The algorithm described is wrong and fails for some test case. Can you find one?

(b) Can you find where did it go wrong?

(c) Fix the algorithm. Analyze the time and space complexity, as well as prove it's correctness.

? Problem

You are given two strings, `Ideal` and `Patient` representing genetic codes.

`Patient` is obtained from `Ideal` by changing exactly k characters.

Determine the indices (with respect to `Ideal`) and values of the modified characters.

(a) Find an algorithm to solve this in $O(n)$ time.